2026-10-25 MySQL索引：上

- 1. MySQL 索引与磁盘 I/O：从查询加速到 Page 管理
  - 1.1 先建立整体认识
    - 1.1.1 索引不是“凭空让 SQL 变快”
    - 1.1.2 CRUD 为什么和内存有关
    - 1.1.3 本课的三层数据通道
  - 1.2 全课知识地图
- 2. 没有索引时，查询为什么会慢
  - 2.1 线性结构与时间复杂度的直觉
    - 2.1.1 没有辅助结构时只能逐条查找
    - 2.1.2 数据量小不代表大数据思路
  - 2.2 提高效率的两条路径
    - 2.2.1 优化算法本身
    - 2.2.2 改变数据的组织方式
    - 2.2.3 为什么有些结构查得快、写得慢
  - 2.3 索引的本质
    - 2.3.1 为搜索重新组织数据
    - 2.3.2 主键为什么会自动带索引
    - 2.3.3 常见索引类型
- 3. 索引性能实验：800 万行数据
  - 3.1 实验数据从哪里来
    - 3.1.1 使用 index.data.sql 生成数据
    - 3.1.2 这张表故意没有主键
  - 3.2 先观察数据
    - 3.2.1 切换到测试数据库
    - 3.2.2 不要直接查询全表
  - 3.3 没有索引时的查询
    - 3.3.1 原始查询
    - 3.3.2 实测耗时
    - 3.3.3 为什么这么慢
  - 3.4 创建索引
    - 3.4.1 使用的语句
    - 3.4.2 创建索引也需要时间
    - 3.4.3 验证索引是否创建成功
  - 3.5 创建索引后的查询
    - 3.5.1 再次执行同一查询

        - 3.5.2 数量级对比
        - 3.5.3 正确理解实验结果
    - 3.6 索引的收益与代价
        - 3.6.1 主要收益
        - 3.6.2 主要代价
- 4. 从磁盘理解数据库 I/O
    - 4.1 为什么必须理解磁盘
        - 4.1.1 数据库最终要持久化
        - 4.1.2 数据库表最终对应文件
    - 4.2 传统机械磁盘的物理结构
        - 4.2.1 盘片、主轴和磁头
        - 4.2.2 磁道、扇区和柱面
    - 4.3 怎样定位一个扇区
        - 4.3.1 CHS 寻址
        - 4.3.2 LBA 逻辑块地址
    - 4.4 随机访问与连续访问
        - 4.4.1 随机访问
        - 4.4.2 连续访问
        - 4.4.3 操作系统会优化 I/O 顺序
    - 4.5 为什么操作系统按 4KB 读取
        - 4.5.1 扇区太小，不适合作为系统 I/O 单位
        - 4.5.2 4KB 与内存页大小对齐
        - 4.5.3 局部性原理
- 5. MySQL、操作系统和磁盘的 I/O 单位
    - 5.1 三种容易混淆的单位
    - 5.4 读取数据的完整路径
        - 5.4.1 MySQL 向操作系统请求 16KB
        - 5.4.2 数据经过两层缓存
    - 5.5 写入和持久化
        - 5.5.1 修改先发生在内存
        - 5.5.2 write 不一定是最终落盘
    - 5.6 为什么更大的单位能提高效率
- 6. InnoDB Page 与 Buffer Pool
    - 6.1 为什么需要管理大量 Page
        - 6.1.1 一个 Page 远远不够
        - 6.1.2 先描述，再组织
    - 6.2 Page 的抽象模型

        - 6.2.1 概念模型
        - 6.2.2 从 Page 对象到 Page 缓冲池
    - 6.3 Buffer Pool 是怎样申请的
        - 6.3.1 启动时预先申请大块空间
        - 6.3.2 为什么一次申请
    - 6.4 一次读取操作的完整过程
        - 6.4.1 查找目标 Page
        - 6.4.2 检查 Buffer Pool
        - 6.4.3 在内存中完成计算
        - 6.4.4 修改后形成脏页
    - 6.5 为什么 16KB Page 可以减少 I/O
        - 6.5.1 一页容纳多条记录
        - 6.5.2 局部性原理再次出现
        - 6.5.3 单个 Page 不能无限增大
- 7. 主键、索引与 Page 的关系
    - 7.1 乱序插入后为什么看起来有序
        - 7.1.1 建立测试表
        - 7.1.2 按乱序主键插入
        - 7.1.3 查询结果为什么按主键排列
        - 7.1.4 不要把观察到的顺序当成 SQL 保证
    - 7.2 为什么页内记录要保持有序
        - 7.2.1 排序是为了更早结束查找
        - 7.2.2 有序并不等于“每次插入都从一个空数组中间搬数据”
    - 7.3 多个 Page 怎样组织
        - 7.3.1 Page 之间用双向链表连接
        - 7.3.2 一个 Page 内部虽有链表，也可以再建目录
    - 7.4 页目录怎样连接大量 Page
        - 7.4.1 目录项由键值和 Page 指针组成
        - 7.4.2 普通页和目录页都是 Page
        - 7.4.3 页目录也可以继续分层
    - 7.5 索引 Page 和普通数据 Page
        - 7.5.1 数据需要 Page
        - 7.5.2 索引也需要 Page
    - 7.6 为什么数据库不直接从磁盘逐条查
- 8. 关键结论与复习
    - 8.1 一张图串起整个主题
    - 8.2 六个必须记住的结论

        - 8.2.1 结构决定算法
        - 8.2.2 索引的主要收益在减少扫描
        - 8.2.3 索引有维护成本
        - 8.2.4 Page 是 MySQL 和操作系统交互的关键单位
        - 8.2.5 Buffer Pool 决定 Page 是否能复用
        - 8.2.6 目录负责缩小 Page 的查找范围
    - 8.3 易错概念对照
        - 8.3.1 Page 和操作系统 Page 不是同一个东西
        - 8.3.2 索引不是只在内存里
        - 8.3.3 查询变快不等于 CPU 计算量减少
        - 8.3.4 16KB 不等于磁盘扇区都是 16KB
        - 8.3.5 不写 ORDER BY 就不要依赖结果顺序
    - 8.4 自测题
        - 8.4.1 为什么 800 万行查询建索引后能从 5 秒降到 0.01 秒?
        - 8.4.2 为什么操作系统一次读取 4KB，而 MySQL 一次使用 16KB Page?
        - 8.4.3 为什么 Buffer Pool 能提前加载后续可能需要的数据?
        - 8.4.4 为什么索引会拖慢插入?
        - 8.4.5 为什么主键乱序插入后查询看起来有序?
        - 8.4.6 为什么多个 Page 不能只靠双向链表查找?
    - 8.5 一页速查

1. MySQL 索引与磁盘 I/O：从查询加速到 Page 管理

> **学习目标：**
>
> 1. 理解为什么没有索引时查询会很慢。
> 2. 理解“结构决定算法”以及索引的收益和代价。
> 3. 读懂 800 万行数据的索引实验。
> 4. 分清磁盘扇区、操作系统数据块和 InnoDB Page。
> 5. 理解 MySQL、操作系统、磁盘之间的数据流动。
> 6. 理解 Buffer Pool 为什么存在，以及 Page 为什么要“先描述，再组织”。

1.1 先建立整体认识

1.1.1 索引不是“凭空让 SQL 变快”

用一句话概括索引的价值：

> 索引的核心工作是提高数据库性能。创建正确的索引后，查询速度可能提高很多倍，尤其是在海量数据中查找少量记录时。

但是索引并不是魔法。它做的是：

1. 以特定数据结构重新组织需要被查找的数据。
2. 减少每次查询需要检查的记录数量。
3. 利用磁盘页和内存缓存，减少真正的 I/O 次数。

因此理解索引需要同时看两个层面：

- **逻辑层面**：怎样组织数据和选择算法。
- **物理层面**：数据怎样从磁盘进入操作系统和 MySQL 内存。

1.1.2 CRUD 为什么和内存有关

> 所有 MySQL 的增、删、查、改操作，最终都是在内存中完成计算的；MySQL 在启动时会预先申请一大块内存，并在合适的时候把数据刷新到磁盘做持久化。

更准确地说，InnoDB 通常遵循这样的过程：

1. 需要访问某个 Page。
2. 如果 Buffer Pool 中已经有这个 Page，就直接使用。
3. 如果没有，就从操作系统和磁盘读取到 Buffer Pool。
4. 在内存中的 Page 上完成查询或修改。
5. 修改后的脏页在后续某个时刻刷回磁盘。

这里有一个重要修正：

> **索引结构本身会持久化在磁盘文件中，并不是只在内存中存在。** 所谓“索引在内存中”，指的是索引 Page 被加载进 Buffer Pool 后，CPU 才能对它进行比较和查找。

1.1.3 本课的三层数据通道

```
SQL 请求
   ↓
MySQL Buffer Pool
   ↓ read / write
```

```
操作系统文件缓存
    ↓ 4KB 数据块
磁盘扇区
```

MySQL 是应用层进程，不能绕过操作系统直接操作磁盘。数据每次从硬件到 MySQL，都要经过“磁盘 → 操作系统 → MySQL”这条路径。

1.2 全课知识地图

主题	核心问题	本课结论
索引	为什么要建索引	降低海量数据查找成本
算法与结构	查询为什么能变快	改变数据组织方式，算法也随之变化
性能实验	索引到底能快多少	800 万行查询由约 5.5 秒降到约 0.01 秒
磁盘	数据最终存在哪里	磁盘扇区中，通过文件系统组织成文件
操作系统	怎样访问磁盘	以 4KB 数据块为基本单位
InnoDB	怎样与操作系统交互	以 16KB Page 为基本单位
Buffer Pool	怎样减少重复 I/O	缓存热点 Page，利用局部性原理
页内目录与页目录	大量记录、多个 Page 怎样快速定位	有序组织与分层目录共同减少比较和 Page 读取

2. 没有索引时，查询为什么会慢

2.1 线性结构与时间复杂度的直觉

2.1.1 没有辅助结构时只能逐条查找

假设数据在线性结构中排列，目标是找出某个值，而数据本身没有额外组织：

- 最直接的办法是从头到尾扫描。
- 平均需要检查一半记录，最坏需要检查全部记录。
- 这种查找的时间复杂度是 **O(n)**。

如果表中有 800 万条记录，而目标记录恰好在很靠后的位置，查询就不得不进行大量比较和 Page 读取。

2.1.2 数据量小不代表大数据思路

课堂中的普通示例表只有几行、几十行，建不建索引几乎看不出区别。索引的价值要到数据量足够大、查询又比较频繁时才会明显体现。

因此专门构造了大约 800 万条记录的 `emp` 表，用来观察：

- 未创建索引时的查询耗时。
- 创建索引时的维护成本。
- 创建索引后同一查询的耗时。

2.2 提高效率的两条路径

2.2.1 优化算法本身

提出：

> 算法效率通常受两个因素影响：一是组织数据的方式，二是算法本身。

如果数组中的元素是**有序**的，就可以把线性查找改成二分查找：

```
无序线性表：逐条扫描          O(n)
有序线性表：使用二分查找       O(log n)
平衡搜索树：按树高向下查找     O(log n)
哈希结构：通过哈希函数定位     O(1)
```

在数据不变的情况下，更换算法就可能明显提高查找效率。

2.2.2 改变数据的组织方式

比“优化原有扫描算法”更进一步的方法，是把线性结构改造成新的结构：

- 把有序数组组织成平衡搜索树。
- 把键值映射组织成哈希表。
- 在数据库表之外，为指定列维护一棵搜索树。

结构一旦改变，适用的查找算法也会改变。课堂把这一点概括为：

> **结构决定算法。**

2.2.3 为什么有些结构查得快、写得慢

不同数据结构对 CRUD 的友好程度不同：

结构	查找特点	插入、删除特点
顺序表	有序时适合二分查找	中间插入、删除可能需要移动元素
链表	通常需要遍历	已知位置时插入、删除方便
哈希表	查找通常很快	需要维护哈希结构和冲突处理
搜索树	有序查找较高效	需要维护平衡、旋转或节点调整

用“天下没有免费的午餐”说明：

> 索引通常会提高读性能，但会增加插入、更新、删除的维护成本，还会占用磁盘和内存空间。

2.3 索引的本质

2.3.1 为搜索重新组织数据

可以把索引理解成：

> 数据库按照特定数据结构维护的辅助查找结构，用来快速定位满足条件的记录或 Page。

原始数据仍然保存在表中，索引保存的是便于查找的键值，以及指向实际数据页或记录的定位信息。

2.3.2 主键为什么会自动带索引

> 创建主键时，底层会自动为主键创建主键索引。

InnoDB 使用聚簇索引组织数据，表的数据通常按主键顺序存放在聚簇索引的叶节点中。因此：

- 主键不仅是唯一性约束，也是重要的数据组织依据。
- 按主键查找通常能从根节点逐层定位到目标 Page。
- 设计主键时不应随意选择变化频繁或过长的字段。

2.3.3 常见索引类型

提到数据库中的常见索引包括：

- `PRIMARY KEY` 主键索引：用于唯一标识记录，InnoDB 会围绕主键组织聚簇索引。
- `UNIQUE` 唯一索引：保证索引列值不重复；一个表中可以建立多个唯一索引。
- `INDEX` 普通索引：只负责加快查找，不要求列值唯一，是业务中最常见的一类辅助索引。
- `FULLTEXT` 全文索引：面向文章或大段文本检索；具体建法、查询方式和引擎限制在“下部分”展开。

本部分先建立“索引会改变数据组织方式”的整体认识。B+ 树页结构、聚簇索引、回表、联合索引和索引操作留到后续继续展开。

3. 索引性能实验：800 万行数据

3.1 实验数据从哪里来

3.1.1 使用 `index.data.sql` 生成数据

准备的 `index.data.sql` 使用 MySQL 存储过程和随机函数：

1. 创建 `bt_index` 数据库。
2. 创建一张用于测试的 `emp` 表。
3. 生成随机字符串和随机数值。
4. 插入大约 800 万条员工记录。

关键代码如下，建表语句应保证列顺序和插入列的个数、类型一致：

```
-- 产生随机字符串
delimiter $$
create function rand_string(n INT)
returns varchar(255)
begin
    declare chars_str varchar(100) default
        'abcdefghijklmnopqrstuvwxyzABCDEFJHIJKLMNOPQRSTUVWXYZ';
    declare return_str varchar(255) default '';
    declare i int default 0;

    while i < n do
        set return_str = concat(
```

```
            return_str,
            substring(chars_str, floor(1 + rand() * 52), 1)
        );
        set i = i + 1;
    end while;
    return return_str;
end $$
delimiter ;

-- 产生随机数字
delimiter $$
create function rand_num()
returns int
begin
    declare i int default 0;
    set i = floor(10 + rand() * 500);
    return i;
end $$
delimiter ;

-- 向员工表批量添加数据
delimiter $$
create procedure insert_emp(in start_no int, in max_num int)
begin
    declare i int default 0;
    set autocommit = 0;

    repeat
        set i = i + 1;
        insert into emp values (
            start_no + i,
            rand_string(6),
            'SALESMAN',
            0001,
            curdate(),
            2000,
            400,
            rand_num()
        );
        until i = max_num
    end repeat;

    commit;
end $$
delimiter ;
```

```
-- 添加 8000000 条记录
call insert_emp(100001, 8000000);
```

如果使用课件提供的完整脚本，也可以直接导入：

```
source /path/to/index.data.sql;
```

路径是讲课环境中的 `index.data.sql`；在自己的环境中应把路径替换成文件实际位置。

这份脚本的目的只是构造差异明显、数量足够大的测试数据，不需要把大量随机数据从终端逐行下载后再观察。建索引前后的查询环境应尽量保持一致，才能让耗时差异更有参考意义。

3.1.2 这张表故意没有主键

课件中的测试表没有特别声明主键：

- 没有任何列被设为主键。
- 没有针对 `empno` 创建索引。
- 数据库无法借助主键索引或普通二级索引直接定位记录。

因此按 `empno` 查询时，只能从数据中逐条扫描。

3.2 先观察数据

3.2.1 切换到测试数据库

```
SHOW DATABASES;
USE bt_index;
SHOW TABLES;
```

可以看到测试表 `emp`。

3.2.2 不要直接查询全表

800 万条记录一次全部返回会消耗大量资源和时间，因此课堂使用 `LIMIT` 先观察少量数据：

```sql
SELECT *
FROM emp
LIMIT 5;
```

这一步只是确认数据已经成功导入，并不用于性能比较。

3.3 没有索引时的查询

3.3.1 原始查询

课堂查询员工编号为 `998877` 的记录：

```sql
SELECT *
FROM emp
WHERE empno = 998877;
```

由于 `empno` 没有索引，MySQL 需要进行全表扫描。

3.3.2 实测耗时

环境中的多次测量大约为：

次数	查询耗时
第一次	6.17 秒
第二次	5.51 秒
第三次	5.65 秒

后续几次基本稳定在 5.5 秒左右。

另一轮演示中，同一条无索引查询显示约 `4.93` 秒；具体秒数会有波动，但与本课笔记中的 5.5 秒级结果属于同一数量级。

> 这些数值来自课堂环境，只用于说明数量级差异；具体耗时会受到 CPU、内存、磁盘、Buffer Pool 大小和缓存状态影响。

3.3.3 为什么这么慢

在没有索引的情况下，数据库需要：

1. 从磁盘读取大量 Page。
2. 把 Page 放入 Buffer Pool。
3. 对读入的记录逐条比较 `empno`。
4. 直到找到目标记录或扫描完整个表。

如果读到的 Page 越多，涉及的 I/O 次数和 CPU 比较次数也越多。

3.4 创建索引

3.4.1 使用的语句

```
ALTER TABLE emp ADD INDEX(empno);
```

含义是：在 `emp` 表的 `empno` 列上创建普通索引。

实际项目中建议给索引起明确名字：

```
ALTER TABLE emp
ADD INDEX idx_emp_empno (empno);
```

也可以显式使用：

```
CREATE INDEX idx_emp_empno ON emp(empno);
```

3.4.2 创建索引也需要时间

中创建该索引大约花了 20 多秒。

这是因为 MySQL 不仅要记录“这条列上需要索引”，还要为已有的 800 万条记录批量构建索引结构。

由此可以看到索引的两类成本：

- 一次性成本：首次建索引需要扫描并组织现有数据。
- 长期成本：后续每次插入、更新、删除都可能需要同步维护索引。

3.4.3 验证索引是否创建成功

可以用下面语句检查：

```
SHOW INDEX FROM emp;
```

重点观察：

- 索引所在的列是否为 `empno`。
- 索引名称是什么。
- 索引的基数、顺序和类型。

3.5 创建索引后的查询

3.5.1 再次执行同一查询

```
SELECT *
FROM emp
WHERE empno = 998877;
```

实测查询耗时降到了约 `0.01` 秒，也就是百分之一秒级。

随后换用另一个编号复查：

```
SELECT *
FROM emp
WHERE empno = 123456;
```

更换查询值仍然很快，说明收益来自 `empno` 索引对目标位置的快速定位，而不是某一条数据碰巧被缓存。

3.5.2 数量级对比

查询条件	是否有索引	课堂实测结果
`empno = 998877`	无	约 5.5 - 6.2 秒
`empno = 998877`	有	约 0.01 秒

速度从秒级降到百分之一秒级，说明索引减少了大量无效扫描。

3.5.3 正确理解实验结果

这个实验说明的是：

- 对 `empno` 做等值查询时，`empno` 索引能显著减少查找成本。
- 查询越快，不代表所有查询都会自动变快。
- 如果查询条件使用了其他没有索引的列，仍可能需要全表扫描。
- 如果返回的行数本来很多，索引也可能无法避免大量数据读取。

3.6 索引的收益与代价

3.6.1 主要收益

索引最适合以下场景：

- 海量数据中的少量精确查找。
- 频繁作为查询条件的列。
- 范围查询、排序和连接条件中的关键列。

3.6.2 主要代价

索引并不是越多越好：

- 占用磁盘空间。
- 运行时占用 Buffer Pool。
- 插入记录时，需要同时维护索引。
- 更新索引列时，可能要调整索引结构。
- 删除记录时，也要维护对应索引项。

因此索引设计的目标不是“给所有列都建索引”，而是在查询收益和维护成本之间取得平衡。

4. 从磁盘理解数据库 I/O

4.1 为什么必须理解磁盘

4.1.1 数据库最终要持久化

内存速度快，但断电后内容会丢失。数据库必须把数据持久化到磁盘。

> 数据库中的数据，最终要落到磁盘的扇区中。

因此数据库性能不仅取决于索引算法，还取决于磁盘 I/O 的特性。

4.1.2 数据库表最终对应文件

在 Linux 中，数据库和表最终都以目录、文件的形式存在：

```
数据库 → 目录
表     → 文件及其中的数据页
```

> 找到一个文件的全部数据，本质上就是在磁盘中找到该文件所在的所有扇区。

4.2 传统机械磁盘的物理结构

4.2.1 盘片、主轴和磁头

传统机械磁盘的基本结构包括：

- **盘片**：数据保存在盘片表面，一个磁盘通常有多张盘片。
- **主轴马达**：带动盘片高速旋转。
- **磁头**：在盘面上寻找目标磁道并读写数据。

读取数据时，盘片旋转、磁头移动，都是机械动作，因此磁盘 I/O 通常远慢于内存访问。

> 这是传统机械硬盘的直观模型。SSD 不使用旋转盘片和机械磁头，但“以固定大小数据块访问、随机与连续访问性能不同”的抽象仍然有意义。

4.2.2 磁道、扇区和柱面

概念	含义
磁道	盘片上的同心圆区域
扇区	磁道按角度划分出的最小存储区域
柱面	多个盘面上相同半径的磁道共同组成的柱面

每个盘片通常有正反两面，每一面都有对应磁头。

传统硬盘常见扇区容量为 512 字节，也有 4KB 扇区；同一磁盘的具体扇区大小要以硬件规格为准。

4.3 怎样定位一个扇区

4.3.1 CHS 寻址

传统磁盘使用 CHS 定位：

- **Cylinder**：柱面。
- **Head**：磁头或盘面。
- **Sector**：扇区。

可以先定位柱面，再选择盘面或磁头，最后定位该磁道上的扇区。

4.3.2 LBA 逻辑块地址

操作系统一般不直接使用 CHS，而使用 **LBA（Logical Block Address，逻辑块地址）**。

LBA 把磁盘看成一段连续的、带编号的块：

```
LBA 0, LBA 1, LBA 2, LBA 3 ...
```

磁盘控制器负责把逻辑块地址转换成底层的物理访问位置。

把 LBA 与 CHS 的关系类比为“虚拟地址与物理地址”：

- 上层软件使用统一的逻辑地址。
- 硬件负责把它转换成实际定位方式。

4.4 随机访问与连续访问

4.4.1 随机访问

如果相邻两次 I/O 的地址跨度很大：

- 磁头需要移动到新的磁道。
- 盘片需要旋转到目标扇区。
- 会产生更多机械动作。

这种访问称为随机访问，通常更慢。

4.4.2 连续访问

如果下一次读取的数据紧接着上一次读取的位置：

- 磁头不需要大幅移动。
- 盘片继续旋转就能读到后续扇区。

这种访问称为连续访问，通常效率更高。

4.4.3 操作系统会优化 I/O 顺序

多个进程的 I/O 请求最终都可能交给操作系统。操作系统会对请求进行合并、排序和调度，使底层硬件尽量以更连续的方式读写。

因此同一块磁盘，因访问位置和组织方式不同，也可能表现出不同的性能。

4.5 为什么操作系统按 4KB 读取

4.5.1 扇区太小，不适合作为系统 I/O 单位

如果操作系统每次只读一个 512 字节扇区：

- 读取同样的数据需要更多次 I/O。
- 每次 I/O 都可能伴随寻道和旋转。
- 文件系统代码会与具体硬件扇区大小强耦合。

所以操作系统通常以更大的块为单位访问文件。

4.5.2 4KB 与内存页大小对齐

Linux 常见的系统内存页和文件 I/O 块大小都是 4KB：

```
1 个操作系统 I/O 块 = 4KB
1 个常见系统 Page = 4KB
```

这不是简单巧合，而是综合考虑了：

- 磁盘与内存之间的数据传输效率。
- 文件系统和内存管理的对齐。
- 操作系统与具体磁盘硬件之间的解耦。
- 局部性原理带来的预加载收益。

4.5.3 局部性原理

程序访问数据时通常具有局部性：

- 刚访问过某块数据，之后很可能继续访问它附近的数据。
- 读取 4KB 时，虽然当前可能只用到几十字节或几百字节，但后续数据大概率也会被访问。

一次多读一点，虽然单次传输量增加，但可能减少后续 I/O 次数，从整体上提高效率。

5. MySQL、操作系统和磁盘的 I/O 单位

5.1 三种容易混淆的单位

层级	常见基本单位	作用
磁盘	扇区 512B 或 4KB	硬件存储与读写的最小物理区域之一

层级	常见基本单位	作用
操作系统	块 4KB	文件系统读写与内存页管理的常见单位
InnoDB	Page 16KB	数据页、Buffer Pool 和磁盘文件组织单位

如果按课堂环境中的 512 字节扇区计算：

```
1 个 InnoDB Page = 16KB
                 = 4 个操作系统 4KB 块
                 = 32 个 512B 磁盘扇区
```

5.4 读取数据的完整路径

5.4.1 MySQL 向操作系统请求 16KB

```
MySQL 的 1 个 InnoDB Page = 16KB
操作系统的 4 个 4KB 块 = 16KB
```

MySQL 并不知道底层磁盘当前使用 512B 还是 4KB 扇区。它主要通过 `read` 等系统调用向操作系统请求数据。

逻辑上可以理解为：

```
MySQL：请读取一个 16KB 的 Page
操作系统：拆成 4 个 4KB 数据块读取
磁盘：底层再按扇区完成实际存储和传输
```

5.4.2 数据经过两层缓存

```
磁盘扇区
   ↓
操作系统文件缓存中的 4KB 块
   ↓
MySQL Buffer Pool 中的 16KB Page
   ↓
CPU 在内存中完成比较、修改和计算
```

MySQL 最终获得的是自己的 16KB Page。它对操作系统底层的 4KB 拆分过程并不需要直接管理。

5.5 写入和持久化

5.5.1 修改先发生在内存

当 MySQL 修改某条记录时，通常会：

1. 找到记录所在的 Page。
2. 确保该 Page 已加载到 Buffer Pool。
3. 在内存中的 Page 内修改数据。
4. 把 Page 标记为脏页。
5. 之后按照刷脏策略，将脏页写回文件。

5.5.2 write 不一定是最终落盘

`write` 通常先把数据写入操作系统缓存。真正要求数据持久化时，还需要通过 `fsync` 等接口刷新到存储设备。

可以概括为：

```
MySQL Buffer Pool
   ↓ write
操作系统文件缓存
   ↓ fsync
持久化存储
```

不同存储引擎和日志机制对持久化的具体实现会有所不同，但“先在内存修改，再按策略刷回磁盘”的整体思路是理解索引和 Page 缓存的基础。

5.6 为什么更大的单位能提高效率

> I/O 效率的主要矛盾不是一次读 4KB 还是 16KB，而是 I/O 发生了多少次。

假设需要读取 16KB 数据：

方案	I/O 次数
每次读 1KB	16 次
每次读 4KB	4 次
每次读 16KB	1 次

如果每次 I/O 都需要重新定位，那么减少 I/O 次数通常比单纯增加单次传输量更重要。

预加载一个 16KB Page 后，本次查询可能只需要其中一小部分记录；但后续查询很可能访问同一 Page 的其他记录，于是可以复用已经加载到 Buffer Pool 的数据。

6. InnoDB Page 与 Buffer Pool

6.1 为什么需要管理大量 Page

6.1.1 一个 Page 远远不够

Buffer Pool 往往远大于单个 16KB Page。数据库运行期间，会同时缓存大量 Page。

因此 MySQL 必须回答很多问题：

- 某个 Page 现在是否在内存中？
- 它是否被修改过？
- 最近是否刚被访问？
- 哪些 Page 应该优先淘汰？
- 脏页应该什么时候刷盘？
- 查找时怎样快速找到目标 Page？

这正是课堂强调的管理问题。

6.1.2 先描述，再组织

课堂用六个字概括管理思路：

> **先描述，再组织。**

不能把 Page 只看成一整块没有属性的 16KB 内存。MySQL 需要为每个 Page 保存管理信息，例如：

- Page 的数据内容。
- Page 编号或归属信息。
- 是否已修改。
- 最近访问时间。
- 淘汰优先级。
- 组织 Page 所需的链接信息。

有了描述结构，才能把大量 Page 组织起来统一管理。

6.2 Page 的抽象模型

6.2.1 概念模型

可以用下面的伪代码理解 Page 的管理结构：

```
struct BufferPage {
    PageId id;           // 页编号
    char data[16 * 1024]; // 16KB 数据
    bool dirty;          // 是否为脏页
    uint64_t lastAccess; // 最近访问信息
    BufferPage* prev;    // 组织所需的前后关系
    BufferPage* next;
};
```

这段代码只是帮助理解，并不等同于 InnoDB 的真实结构。真实实现还会包含锁、校验、版本、LRU 组织等复杂信息。

6.2.2 从 Page 对象到 Page 缓冲池

可以把 Buffer Pool 理解成大量 Page 对象的集合：

```
Buffer Pool
├── Page 1
├── Page 2
```

```
├── Page 3
└── ...
```

缓存中的 Page 不是永远保留：

- 热点 Page 会被频繁访问。
- 冷门 Page 可能在空间不足时被淘汰。
- 脏页在淘汰或达到刷新条件时需要写回磁盘。
- 未修改的干净页可以直接释放，因为磁盘上已有原始数据。

把这些淘汰和整理思路类比为 `LRU`，重点在于理解“最近很少使用的 Page 更可能被换出”，而不要求死记具体链表实现。

6.3 Buffer Pool 是怎样申请的

6.3.1 启动时预先申请大块空间

MySQL 在启动并运行后，会预先申请一块较大的内存区域作为 Buffer Pool，用 Page 级别的缓存承接磁盘与 CPU 之间的数据流动。

环境中默认大小为约 128MB。可以通过配置项观察：

```
innodb_buffer_pool_size=128M
```

> 128MB 是演示默认情况，不代表所有 MySQL 版本和部署环境都相同。生产环境通常需要结合内存容量、数据集大小和并发量调整。

6.3.2 为什么一次申请

如果每次 I/O 都重新申请 16KB 小内存：

- 内存分配和释放有开销。
- 缓存无法在多次查询之间复用。
- 局部性原理带来的优势无法发挥。

预先申请大块空间并自行管理 Page，可以让热点数据长期驻留，减少频繁分配和磁盘访问。

6.4 一次读取操作的完整过程

6.4.1 查找目标 Page

执行查询时，MySQL 根据索引或表的组织方式，先确定可能需要访问的 Page。

6.4.2 检查 Buffer Pool

如果 Page 已在 Buffer Pool：

```
直接使用内存中的 Page，不发生新的磁盘读取
```

如果 Page 不在 Buffer Pool：

```
向操作系统请求 Page
   ↓
操作系统从磁盘读取
   ↓
数据进入操作系统缓存
   ↓
16KB Page 被装入 Buffer Pool
```

6.4.3 在内存中完成计算

CPU 比较、过滤、排序和修改都必须针对内存中的数据进行。数据库不能围绕磁盘上的单个字节做计算。

6.4.4 修改后形成脏页

如果本次操作修改了 Page：

- Page 与磁盘上的旧版本暂时不一致。
- 该 Page 被标记为脏页。
- 后台或特定时机再把 Page 刷回磁盘。

6.5 为什么 16KB Page 可以减少 I/O

6.5.1 一页容纳多条记录

一个 Page 有 16KB，通常会包含多条记录。如果目标记录在同一个 Page 中：

- 第一次读取该 Page 时发生一次 I/O。
- 后续访问同页记录可以直接在 Buffer Pool 中完成。
- 不需要为每条记录分别访问磁盘。

6.5.2 局部性原理再次出现

如果数据库正在访问某一行，接下来很可能访问同一页附近的行：

- 范围查询会连续读取附近记录。
- 索引顺序扫描会访问相邻 Page。
- 同一业务对象的数据经常被一起读取。

预加载一个 Page，相当于提前把周边数据带入内存，命中时可以显著减少 I/O 次数。

6.5.3 单个 Page 不能无限增大

Page 如果过大：

- 一次读取会浪费更多带宽。
- 即使只访问少量数据，也要搬运大量无用内容。
- Buffer Pool 中可缓存的 Page 数量减少。

16KB 是数据页大小、缓存效率和内存对齐之间的工程取舍。

7. 主键、索引与 Page 的关系

7.1 乱序插入后为什么看起来有序

7.1.1 建立测试表

创建了一张带主键的 `user` 表：

```
CREATE TABLE user (
    id INT PRIMARY KEY,
    age INT NOT NULL,
    name VARCHAR(16) NOT NULL
) ENGINE=InnoDB;
```

查看表结构：

```
DESC user;
SHOW CREATE TABLE user;
```

可以看到存储引擎是 InnoDB，`id` 是主键。

7.1.2 按乱序主键插入

故意按 `3、4、2、5、1` 的顺序插入数据。第一条记录是：

```
INSERT INTO user (id, age, name)
VALUES (3, 18, '杨过');
```

其余四行插入如下：

```
INSERT INTO user (id, age, name) VALUES
    (4, 16, '小龙女'),
    (2, 26, '黄蓉'),
    (5, 36, '郭靖'),
    (1, 56, '欧阳锋');
```

7.1.3 查询结果为什么按主键排列

执行：

```
SELECT *
FROM user;
```

观察到结果按 `1、2、3、4、5` 显示，而插入顺序是 `3、4、2、5、1`。

原因是：

- `id` 是 InnoDB 主键。
- 主键对应聚簇索引。
- InnoDB 的数据记录组织在聚簇索引中，通常按主键顺序存放。
- 当发生 Page 分裂和记录整理后，按索引顺序扫描更可能看到主键有序的结果。

7.1.4 不要把观察到的顺序当成 SQL 保证

虽然实验结果是按主键显示，但 SQL 标准并不保证“不写 `ORDER BY` 时一定按主键返回”。

需要确定顺序时，应该明确写出：

```
SELECT *
FROM user
ORDER BY id;
```

正确理解是：

> InnoDB 的数据组织方式会影响实际读取顺序，但业务结果不能依赖未声明的返回顺序。

7.2 为什么页内记录要保持有序

7.2.1 排序是为了更早结束查找

Page 中的记录仍然通过前后指针连接，具有链表的增删特点：如果无序，定位某条记录就可能必须从头逐条比较。按主键维护顺序后：

- 页内比较遵循单调顺序，不会出现大量已经越过的无效区间。
- 达到目标键或范围边界后，可以提前结束查找。
- 后续才能利用二分式目录定位，而不必从页头开始扫描。

因此，InnoDB 插入记录时维护有序结构，是在为查询效率付出维护成本。

7.2.2 有序并不等于“每次插入都从一个空数组中间搬数据”

这里的有序是 InnoDB 页内组织与索引结构共同维持的结果，不应简单套用普通数组“插入时移动全部元素”的模型。真正实现还涉及页内空闲空间、记录槽位、页分裂与索引节点维护；本部分只需抓住一个因果链：

```
维持有序
    ↓
可以用目录缩小查找范围
    ↓
减少比较次数和需要读取的 Page
```

7.3 多个 Page 怎样组织

7.3.1 Page 之间用双向链表连接

一页固定为 16KB，存不下整张表，所以表通常由多个 Page 构成。课件用 `prev` 和 `next` 指针把 Page 连成双向链表：

```
Page 1 ⇄ Page 2 ⇄ Page 3 ⇄ Page 4
```

这样做有两个直接效果：

- 已知相邻页的位置时，可以顺着指针继续读取。
- 范围查询可以按顺序访问后续 Page，而不必回到整张表的起点。

不过，只有 Page 间的链表仍然不够。若目标记录在很后面的 Page，系统仍可能逐页加载并在 Page 之间线性遍历，这会产生大量 I/O。

7.3.2 一个 Page 内部虽有链表，也可以再建目录

单页中记录用链表组织。若查找 `id=4`，没有目录就可能从第一条开始依次比较。建立页内目录后，可以先通过目录项缩小起始范围，再从合适位置继续查找。

可以用书来理解：

- 从第一页翻到目标章节，慢。
- 先查目录，直接翻到大致页码，再阅读目标内容，快。
- 目录多占了纸张，却显著减少了翻找成本。

页内目录也是“以额外空间换查找时间”的具体体现。

7.4 页目录怎样连接大量 Page

7.4.1 目录项由键值和 Page 指针组成

Page 数量变多后，只优化页内查找不够，因为系统仍要花很多 I/O 在“应该先读哪一页”上。课件提出用页级目录来管理 Page，每个目录项主要包含：

```
目标 Page 中的最小键值 + 指向该 Page 的指针
```

查找时先比较目录中的键值，再沿指针找到对应 Page。这样可以把一个大范围内的 Page 排除掉，只加载可能与目标有关的少量 Page。

7.4.2 普通页和目录页都是 Page

强调，目录页本质上也是 Page：

- 普通数据页保存用户记录。
- 目录页保存下级 Page 的最小键值和地址。
- 页内目录管理的是“行”的定位。
- 页级目录管理的是“Page”的定位。

这种分层把一次大范围线性扫描变成“少量目录页比较 + 少量数据页读取”。

7.4.3 页目录也可以继续分层

如果普通 Page 很多，只用一个目录页也会再次变长，甚至无法放进单个 Page。解决办法还是同一思路：给目录页再建上一层目录。

```
上层目录页
   ↓ 选定一段键值范围
下层目录页
   ↓ 选定目标普通 Page
数据 Page
   ↓ 页内目录 + 顺序比较
目标记录
```

层数增加后，单次查找读取的 Page 数仍然可能很少。这里已经出现了多级索引的雏形；课件下一页把它命名为 B+ 树，具体节点结构和索引操作放到“下部分”继续整理。

7.5 索引 Page 和普通数据 Page

7.5.1 数据需要 Page

InnoDB 表中的记录按 Page 组织，读取一条记录通常要先把所在 Page 装入 Buffer Pool。

7.5.2 索引也需要 Page

索引并不是一条条永久放在内存里的独立记录。索引同样有树结构、内部节点和叶子节点，这些节点也由 Page 组织。

因此，一个查询可能涉及：

1. 读取索引 Page，找到目标键。
2. 根据索引定位实际数据 Page。
3. 把数据 Page 加载进 Buffer Pool。

不同类型索引是否“回表”取决于索引和查询字段，后续学习 B+ 树与覆盖索引时会继续展开。

7.6 为什么数据库不直接从磁盘逐条查

因为 CPU 计算的前提是数据已经在内存中。磁盘的最小访问单位和速度特性都不适合逐条处理：

- 磁盘 I/O 远慢于内存访问。
- 每条记录独立读盘会产生大量 I/O。
- 16KB Page 能一次带入多条相关记录。
- Buffer Pool 能让 Page 在多次查询之间复用。

所以数据库的核心优化之一，就是让索引定位更准、让 Page 命中率更高、让 I/O 次数更少。

8. 关键结论与复习

8.1 一张图串起整个主题

```
没有索引
    ↓
全表扫描大量 Page
    ↓
I/O 次数多，查询慢

创建索引
    ↓
数据组织方式改变
    ↓
通过有序结构、页内目录和页目录快速定位
    ↓
减少扫描范围和 I/O 次数
    ↓
查询速度显著提高
```

8.2 六个必须记住的结论

8.2.1 结构决定算法

索引的核心不是单纯“换一种查找代码”，而是改变数据组织方式，使更高效的查找算法成为可能。

8.2.2 索引的主要收益在减少扫描

对海量数据中的少量目标记录，索引可以显著减少无效记录比较和 Page 读取。

8.2.3 索引有维护成本

索引会占空间，并增加增删改的维护工作。索引不是越多越好。

8.2.4 Page 是 MySQL 和操作系统交互的关键单位

InnoDB Page 通常为 16KB；操作系统常用 4KB 块；磁盘常见 512B 或 4KB 扇区。

8.2.5 Buffer Pool 决定 Page 是否能复用

Page 命中 Buffer Pool 时不需要再次访问磁盘。预加载、局部性和 LRU 管理共同帮助减少 I/O 次数。

8.2.6 目录负责缩小 Page 的查找范围

Page 内目录缩小记录查找范围，页级目录缩小目标 Page 的查找范围；目录继续分层后，才能让数据量增长时不必逐页线性扫描。

8.3 易错概念对照

8.3.1 Page 和操作系统 Page 不是同一个东西

名称	常见大小	所属层级
磁盘扇区	512B 或 4KB	硬件
操作系统页或块	4KB	操作系统
InnoDB Page	16KB	MySQL 存储引擎

8.3.2 索引不是只在内存里

- 索引结构持久化在数据库文件中。
- 访问中的索引 Page 缓存在 Buffer Pool。
- 内存中的修改最终要刷新到磁盘。

8.3.3 查询变快不等于 CPU 计算量减少

索引减少的是无效数据定位和 I/O 扫描。最终仍要读取目标 Page、比较键值并在内存中完成计算。

8.3.4 16KB 不等于磁盘扇区都是 16KB

16KB 是 InnoDB Page 的大小，不代表磁盘每读取一次就一定只有 16KB，也不代表硬件扇区也是 16KB。

8.3.5 不写 ORDER BY 就不要依赖结果顺序

主键聚簇结构会影响实际读取顺序，但业务代码必须显式声明排序要求。

8.4 自测题

8.4.1 为什么 800 万行查询建索引后能从 5 秒降到 0.01 秒?

因为索引让 MySQL 能通过索引 Page 快速定位 `empno=998877` 的位置，而不必逐页扫描全部 800 万行。

8.4.2 为什么操作系统一次读取 4KB，而 MySQL 一次使用 16KB Page?

操作系统用 4KB 对齐系统和文件 I/O；InnoDB 用 16KB Page 组织记录和索引。MySQL 向操作系统请求 16KB 时，操作系统可以在内部拆成 4 个 4KB 块读取。

8.4.3 为什么 Buffer Pool 能提前加载后续可能需要的数据?

因为数据访问具有局部性。目标记录附近的数据之后很可能被访问，预加载整个 Page 可以减少后续 I/O。

8.4.4 为什么索引会拖慢插入?

插入记录不仅要修改数据 Page，还可能要更新一个或多个索引 Page。索引越多、维护成本可能越高。

8.4.5 为什么主键乱序插入后查询看起来有序?

因为 InnoDB 使用主键聚簇索引组织数据，但查询结果的确定顺序仍然需要 `ORDER BY` 显式保证。

8.4.6 为什么多个 Page 不能只靠双向链表查找?

双向链表适合顺着相邻页继续访问，适合顺序扫描；但不知道目标在哪一页时，仍可能逐页读取和线性比较。页级目录用最小键值与 Page 指针缩小范围，再逐层定位到目标 Page。

8.5 一页速查

```
-- 查看 InnoDB Page 大小
SHOW GLOBAL STATUS LIKE 'innodb_page_size';

-- 创建普通索引
CREATE INDEX idx_emp_empno ON emp(empno);
```

```sql
-- 通过 ALTER TABLE 创建索引
ALTER TABLE emp ADD INDEX idx_emp_empno (empno);

-- 查看索引
SHOW INDEX FROM emp;

-- 查看表结构和存储引擎
SHOW CREATE TABLE user;
```

索引原理：
改变数据组织方式 → 快速定位记录/Page → 减少扫描与 I/O

物理单位：
通常 512B 磁盘扇区
4KB 操作系统块
16KB InnoDB Page

缓存流程：
磁盘 → 操作系统缓存 → Buffer Pool → 内存计算 → 按策略刷盘

索引定位：
有序记录 → 页内目录 → 页级目录 → 目标 Page → 目标记录

性能关键：
减少 I/O 次数，提高 Page 命中率，利用局部性原理